

Teoria da Computação e Linguagens Formais

Profa. Dra. Jaqueline Brigladori Pugliesi



Conceitos Básicos

Definições (cont.)

- Procedimento é uma sequência finita de instruções que podem ser executadas por um agente computacional (humano ou não).
- Um procedimento que sempre termina é chamado Algoritmo.
- Um Programa é uma sequência de símbolos de uma linguagem.

Definições (cont.)

- Linguagem:
 - Forma precisa de expressar problemas, permitindo um desenvolvimento formal adequado ao estudo da computabilidade.
 - Envolve uso de um alfabeto e regras de formação de palavras e combinação dessas palavras.
- Alfabeto (Σ): conjunto finito de zero ou mais símbolos
 - Ex:
 - $\{a,b,c\}$ alfabeto constituído de a, b e c.
 - $\emptyset$ conjunto vazio
 - $\{a,b,c, \dots\}$ não é um alfabeto

Definições (cont.)

- Palavra ou cadeia de símbolos (w):
 - Uma sequência de zero ou mais símbolos justapostos
 - Ex:
 - ε palavra vazia
 - Σ^* conjunto de todas as palavras possíveis formadas a partir do alfabeto
 - $\Sigma^+ = \Sigma^* - \varepsilon$

Definições (cont.)

- Comprimento ou tamanho da palavra $|w|$
 - Número de símbolos que compõem a palavra
 - Ex:
 - $w = aabc \quad |w| = 4$
- Prefixo
 - Qualquer sequência inicial de símbolos da palavra
 - $w = aabc$ prefixos : $\varepsilon, a, aa, aab, aabc$
- Sufixo
 - Qualquer sequência final de símbolos da palavra
 - $w = aabc$ sufixos: $\varepsilon, c, bc, abc, aabc$

Definições (cont.)

- Subpalavra
 - Qualquer sequência de símbolos contínuos na palavra. Ex:
 - $w = aabc$
 - subpalavra = todos os prefixos e sufixos, além de: ab .
- Linguagem Formal
 - É um conjunto de palavras sobre um alfabeto. Ex:
 - $\Sigma = \{a,b,c\}$
 - $LF = \{w \mid w \text{ pertence a } \Sigma^* \text{ e tem sufixo igual a } c\}$

Definições (cont.)

- Concatenação de palavras
 - Resulta da justaposição de uma palavra com outra. Ex:
 - $w = a$
 - $v = ab$
 - $wv = aab$
 - Não necessariamente fechada em L
 - A concatenação de duas palavras de um linguagem L não necessariamente produzirá uma palavra pertencente a L
 - É associativa
 - $vwt = v(wt) = (vw)t$
 - Possui elemento neutro
 - $\varepsilon w = w = w\varepsilon$

Definições (cont.)

- Concatenação sucessiva
 - Justaposição de uma palavra com ela mesma n vezes
 - $w^n = \text{wwwwww} \dots$ n concatenações de w
 - $w^0 = \varepsilon$
 - $w^n = w^{n-1} w$, para $n > 0$
 - Se $w = \text{aab}$ $w^3 = \text{aabaabaab}$

Gramática

- Gramática é uma quádrupla ordenada $G = \{V, T, P, S\}$ onde:
 - V ou V_N = conjunto finito de símbolos não terminais
 - T ou V_T = conjunto finito de símbolo terminais disjunto de V .
 - P = conjunto finito de pares, denominados regras de produção
 - Uma regra de produção é da forma:
 - $A \rightarrow B$
 - $A \rightarrow B \mid C \mid \dots \mid F$
 - A partir de uma regra de produção podemos formar uma palavra
 - S = símbolo não terminal inicial.

Gramática (cont.)

- A partir de uma gramática podemos gerar uma Linguagem, que será composta por todas as palavras formadas por símbolos terminais derivadas da gramática

Exemplo de Gramática

$G = \{V, T, P, S\}$ onde:

$V = \{S, D\}$

$T = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$

$P = \{ S \rightarrow D,$

$S \rightarrow SD,$

$D \rightarrow 0 \mid 1 \mid 2 \mid \dots \mid 9 \}$

Gramática que produz a linguagem formada pelos números naturais.

Exercício

Considere a gramática

$G = (\{S, X, Y, A, B, F\}, \{a, b\}, P, S)$, em que:

$P = \{$

$S \rightarrow XY$

$X \rightarrow XaA \mid XbB \mid F$

$Aa \rightarrow aA, Ab \rightarrow bA, AY \rightarrow Ya,$

$Ba \rightarrow aB, Bb \rightarrow bB, BY \rightarrow Yb,$

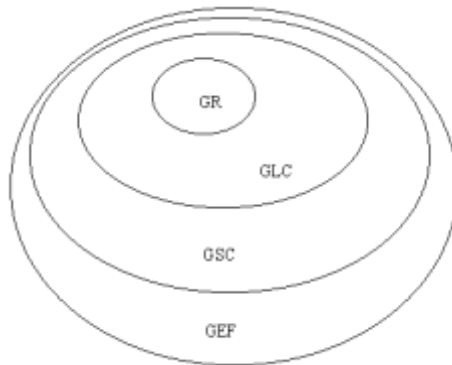
$Fa \rightarrow aF, Fb \rightarrow bF, FY \rightarrow \varepsilon$

$\}$

Mostre os passos de derivação da palavra **baba**

Autômatos e Linguagens Formais

Hierarquia de Chomsky



GR = Gramáticas Regulares

GLC = Gramáticas Livres de Contexto

GSC = Gramáticas Sensíveis ao Contexto

GEF = Gramáticas com Estrutura de Frase

Gramáticas com Estrutura de Frase ou Tipo 0

- São aquelas às quais nenhuma limitação é imposta.
- Para gramáticas do tipo 0, as produções são todas da forma

$$\alpha \rightarrow \beta \quad \text{onde} \quad \alpha \in (V_n \cup V_t)^+ \quad \text{e} \quad \beta \in (V_n \cup V_t)^*$$

Exemplo:

$$G = (\{A, B, C\}, \{a, b\}, P, A)$$

$$P: A \rightarrow BC$$

$$BC \rightarrow CB$$

$$B \rightarrow b$$

$$C \rightarrow a$$

$$L(G) = \{ba, ab\}$$

Gramáticas Sensíveis ao Contexto ou Tipo I

- É imposta a restrição de que nenhuma substituição possa reduzir o comprimento da forma sentencial.

$$\alpha \rightarrow \beta, \text{ tal que } |\alpha| \leq |\beta| \text{ onde } \alpha, \beta \in (V_n \cup V_t)^+$$

Exemplo:

$$G = (\{S, B, C\}, \{a, b, c\}, P, S)$$

$$P: S \rightarrow aSBC$$

$$S \rightarrow aBC$$

$$CB \rightarrow BC$$

$$aB \rightarrow ab$$

$$bB \rightarrow bb$$

$$bC \rightarrow bc$$

$$cC \rightarrow cc$$

Gramáticas Livres de Contexto ou Tipo 2

- Restringe as produções à forma:

$$A \rightarrow \alpha \quad \text{onde } A \in V_n, \alpha \in (V_n \cup V_t)^*$$

- Exemplo:

$$G = (\{S\}, \{a, +, *, (,)\}, P, S)$$

$$P = S \rightarrow S * S$$

$$S \rightarrow S + S$$

$$S \rightarrow (S)$$

$$S \rightarrow a$$

$L(G)$ = conjunto das expressões aritméticas envolvendo $*, +, (,), a$.

Um exemplo de cadeia formada por esta gramática é

$$a * (a + a)$$

Gramáticas Regulares ou Tipo 3

- As produções são restritas às formas seguintes:

$$A \rightarrow aB$$

ou

$$A \rightarrow b$$

onde $A, B \in V_n$, $a \in V_t$ e $b \in V_t \cup \{\lambda\}$

- Exemplo:

$$G = (\{S\}, \{a, b\}, P, S)$$

$$P = S \rightarrow aS$$

$$S \rightarrow b$$

$$L(G) = \{a^n b, n=0\}$$

Linguagens $\Rightarrow$ Reconhecedores

Linguagem	Gramática	Reconhecedor
tipo 0: Conjuntos Recursivamente enumeráveis	tipo 0: Gramáticas com Estrutura de Frase	Máquinas de Turing
tipo 1: Sensíveis ao Contexto	tipo 1: Gramáticas Sensíveis ao Contexto	Máquinas de Turing com memória limitada
tipo 2: Livres de Contexto	tipo 2: Gramáticas Livres de Contexto	Autômatos à Pilha
tipo 3: Conjuntos Regulares	tipo 3: Gramáticas Regulares	Autômatos Finitos

Gramática - mini Pascal

<program> ::= program <identifier> ; <block> .

*<block> ::= <variable declaration part>
 <procedure declaration part>
 <statement part>*

*<variable declaration part> ::= <empty> |
 var <variable declaration> ;
 { <variable declaration> ; }*

<variable declaration> ::= <identifier> { , <identifier> } : <type>

<type> ::= <simple type> | <array type>

*<array type> ::= **array** [<index range>] **of** <simple type>*

<index range> ::= <integer constant> .. <integer constant>

<simple type> ::= <type identifier>

<type identifier> ::= <identifier>

<procedure declaration part> ::= { <procedure declaration> ; }

*<procedure declaration> ::= **procedure** <identifier> ; <block>*

<statement part> ::= <compound statement>

*<compound statement> ::= **begin** <statement> { ; <statement> } **end***

<statement> ::= <simple statement> | <structured statement>

*<simple statement> ::= <assignment statement> | <procedure statement> |
 <read statement> | <write statement>*

<assignment statement> ::= <variable> := <expression>

<procedure statement> ::= <procedure identifier>

<procedure identifier> ::= <identifier>

*<read statement> ::= **read** (<input variable> { , <input variable> })*

<input variable> ::= <variable>

*<write statement> ::= **write** (<output value> { , <output value> })*

<output value> ::= <expression>

*<structured statement> ::= <compound statement> | <if statement> |
 <while statement>*

*<if statement> ::= **if** <expression> **then** <statement> |
 if <expression> **then** <statement> **else** <statement>*

*<while statement> ::= **while** <expression> **do** <statement>*

$\langle \text{expression} \rangle ::= \langle \text{simple expression} \rangle \mid$
 $\quad \langle \text{simple expression} \rangle \langle \text{relational operator} \rangle \langle \text{simple expression} \rangle$
 $\langle \text{simple expression} \rangle ::= \langle \text{sign} \rangle \langle \text{term} \rangle \{ \langle \text{adding operator} \rangle \langle \text{term} \rangle \}$
 $\langle \text{term} \rangle ::= \langle \text{factor} \rangle \{ \langle \text{multiplying operator} \rangle \langle \text{factor} \rangle \}$
 $\langle \text{factor} \rangle ::= \langle \text{variable} \rangle \mid \langle \text{constant} \rangle \mid (\langle \text{expression} \rangle) \mid \text{not } \langle \text{factor} \rangle$

 $\langle \text{relational operator} \rangle ::= = \mid < \mid < = \mid > = \mid >$
 $\langle \text{sign} \rangle ::= + \mid - \mid \langle \text{empty} \rangle$
 $\langle \text{adding operator} \rangle ::= + \mid - \mid \text{or}$
 $\langle \text{multiplying operator} \rangle ::= * \mid \text{div} \mid \text{and}$

 $\langle \text{variable} \rangle ::= \langle \text{entire variable} \rangle \mid \langle \text{indexed variable} \rangle$
 $\langle \text{indexed variable} \rangle ::= \langle \text{array variable} \rangle [\langle \text{expression} \rangle]$
 $\langle \text{array variable} \rangle ::= \langle \text{entire variable} \rangle$
 $\langle \text{entire variable} \rangle ::= \langle \text{variable identifier} \rangle$
 $\langle \text{variable identifier} \rangle ::= \langle \text{identifier} \rangle$

Lexical grammar

$\langle \text{constant} \rangle ::= \langle \text{integer constant} \rangle \mid \langle \text{character constant} \rangle \mid \langle \text{constant identifier} \rangle$
 $\langle \text{constant identifier} \rangle ::= \langle \text{identifier} \rangle$
 $\langle \text{identifier} \rangle ::= \langle \text{letter} \rangle \{ \langle \text{letter or digit} \rangle \}$
 $\langle \text{letter or digit} \rangle ::= \langle \text{letter} \rangle \mid \langle \text{digit} \rangle$
 $\langle \text{integer constant} \rangle ::= \langle \text{digit} \rangle \{ \langle \text{digit} \rangle \}$
 $\langle \text{character constant} \rangle ::= \text{'< any character other than ' >'} \mid \text{'\"'}$
 $\langle \text{letter} \rangle ::= \text{a} \mid \text{b} \mid \text{c} \mid \text{d} \mid \text{e} \mid \text{f} \mid \text{g} \mid \text{h} \mid \text{i} \mid \text{j} \mid \text{k} \mid \text{l} \mid \text{m} \mid \text{n} \mid \text{o} \mid$
 $\quad \text{p} \mid \text{q} \mid \text{r} \mid \text{s} \mid \text{t} \mid \text{u} \mid \text{v} \mid \text{w} \mid \text{x} \mid \text{y} \mid \text{z} \mid \text{A} \mid \text{B} \mid \text{C} \mid$
 $\quad \text{D} \mid \text{E} \mid \text{F} \mid \text{G} \mid \text{H} \mid \text{I} \mid \text{J} \mid \text{K} \mid \text{L} \mid \text{M} \mid \text{N} \mid \text{O} \mid \text{P}$
 $\quad \text{Q} \mid \text{R} \mid \text{S} \mid \text{T} \mid \text{U} \mid \text{V} \mid \text{W} \mid \text{X} \mid \text{Y} \mid \text{Z}$
 $\langle \text{digit} \rangle ::= 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9$
 $\langle \text{special symbol} \rangle ::= + \mid - \mid * \mid = \mid < \mid < = \mid > \mid > = \mid$
 $\quad (\mid) \mid [\mid] \mid := \mid . \mid , \mid ; \mid : \mid \dots \mid \text{div} \mid \text{or} \mid$
 $\quad \text{and} \mid \text{not} \mid \text{if} \mid \text{then} \mid \text{else} \mid \text{of} \mid \text{while} \mid \text{do} \mid$
 $\quad \text{begin} \mid \text{end} \mid \text{read} \mid \text{write} \mid \text{var} \mid \text{array} \mid$
 $\quad \text{procedure} \mid \text{program}$
 $\langle \text{predefined identifier} \rangle ::= \text{integer} \mid \text{Boolean} \mid \text{true} \mid \text{false}$

Gramática - miniC

Function	→	Type identifier (ArgList) CompoundStmt
ArgList	→	Arg ArgList , Arg
Arg	→	Type identifier
Declaration	→	Type IdentList ;
Type	→	int float
IdentList	→	identifier , IdentList identifier
Stmt	→	ForStmt WhileStmt Expr ; IfStmt CompoundStmt Declaration ;
ForStmt	→	for { Expr ; OptExpr ; OptExpr } Stmt

OptExpr	→	Expr ε
WhileStmt	→	while (Expr) Stmt
IfStmt	→	if (Expr) Stmt ElsePart
ElsePart	→	else Stmt ε
CompoundStmt	→	{ StmtList }
StmtList	→	StmtList Stmt ε
Expr	→	identifier = Expr Rvalue
Rvalue	→	Rvalue Compare Mag Mag
Compare	→	== < > <= >= !=
Mag	→	Mag + Term Mag - Term Term
Term	→	Term * Factor Term / Factor Factor
Factor	→	(Expr) - Factor + Factor identifier number

Gramática – mini Java

Goal = MainClass, { ClassDeclaration }, EOF;

MainClass = "class", Identifier, "{", "public", "static", "void", "main", "(",
"String", "[", "]", Identifier, ")", "{", Statement, "}", "};"

ClassDeclaration = "class", Identifier, { "extends", Identifier }, "{", {
VarDeclaration }, { MethodDeclaration } "};"

VarDeclaration = Type, Identifier, ";;"

MethodDeclaration = "public", Type, Identifier, "(", { Type, Identifier, { ":", Type,
Identifier }, }, ")", "{", { VarDeclaration }, { Statement },
"return", Expression, ";;", "};"

Type = "int", "[", "]"

| "boolean"

| "int"

| Identifier

;

Statement = "{", { Statement }, "}"

| "if", "(", Expression, ")", Statement, "else", Statement

| "while", "(", Expression, ")", Statement

| "System.out.println", "(", Expression, ")", ";;"

| Identifier, "=", Expression, ";;"

| Identifier, "[", Expression, "]", "=", Expression, ";;"

;

```

Expression = Expression , ( "&&" | "<" | "+" | "-" | "*" ) , Expression
| Expression , "[", Expression , "]"
| Expression , ".", "length"
| Expression , ".", Identifier , "(", [ Expression { ",", Expression }
], ")"
| IntegerLiteral
| "true"
| "false"
| Identifier
| "this"
| "new", "int", "[", Expression , "]"
| "new", Identifier , "(", ")"
| "!", Expression
| "(", Expression , ")"
,

```

Teoria da Computação

Teoria da Computação

- A Teoria da Computação:
 - Compreende propriedades matemáticas fundamentais do hardware, do software e de aplicações de computadores.
 - Estuda o que pode e o que não pode ser computado, quão rapidamente, com quanto de memória e sobre que tipo de modelo computacional.

Teoria da Computação na prática

- A Teoria da Computação é relevante na prática:
 - Para projetar uma nova linguagem de programação para uma aplicação especializada.
 - Para lidar com a busca por cadeias e casamento de padrões.
 - Para justificar a escolha de uma solução computacional em detrimento de outra.

Áreas da Teoria da Computação

- Teoria da Complexidade
- Teoria da Computabilidade
- Teoria dos Autômatos

Teoria da Complexidade

- Estuda como evidenciar os problemas computacionalmente difíceis e problemas computacionalmente fáceis.
 - Exemplo:
 - Problema 1: ordenar em ordem crescente uma lista de números.
 - Problema 2: escalonar para uma universidade as suas mil aulas, de forma que duas aulas não podem ocorrer na mesma sala no mesmo tempo.
 - Encontrar o melhor escalonamento pode durar séculos até mesmo para um supercomputador.

Teoria da Computabilidade

- Intimamente relacionada com a Teoria da Complexidade.
- Tem por objetivo classificar os problemas como solúveis e não solúveis.
 - Ex: determinar se um procedimento é algoritmo ou não (não é solúvel).

Teoria dos Autômatos

- Lida com definições e propriedades de modelos matemáticos.
- Tais modelos matemáticos são usados em:
 - processamento de texto,
 - construção de compiladores,
 - linguagens de programação,
 - inteligência artificial.

